

SPOT CHECK

SKATESPOTS FIELD MANUAL · Nº05

STATE, TYPES & OWNERSHIP

Ten ideas pulled out of today's session — finishing the add-spot form, building `Badge`, `Indicator`, `StarRating`, and the filter bar. Each chapter is What / How / Why, plus the keywords to know and one idea worth carrying into the next thing you build.

10 chapters

Glossary

Self-test

Phase 4 — Remaining UI

► CONTENTS (10 CHAPTERS + APPENDICES)

01 Custom `<select>` styling

● CONCEPT

● GOTCHA

KEYWORDS

`replaced element`

`appearance: none`

`data URI`

— WHAT

Making a native `<select>` match a dark custom design instead of the browser's default box-and-arrow.

— HOW

`appearance: none` strips all native chrome, including the arrow. A `background-image` — a small SVG chevron embedded as a data URI — draws a replacement arrow

back in, positioned with `background-position`. Extra `padding-right` keeps option text clear of it.

```
.select {
  appearance: none;
  background-image: url("data:image/svg+xml,...chevron...");
  background-repeat: no-repeat;
  background-position: right 12px center;
  padding: 10px 36px 10px 12px; /* room for the arrow */
}
```

— WHY

A `<select>` is a **replaced element** — the browser renders its whole box as one opaque widget. You can't put a child `<span>` or a `::after` pseudo-element inside it to fake an arrow, because there's no "inside" to reach. Once native chrome is gone, `background-image` is the one styling channel still open.

◆ IDEA TO CONSIDER

This pattern generalizes past one dropdown: any "replaced element" — `<video>` controls, `<input type="file">`, `<input type="range">` — resists normal styling the same way, for the same reason. Recognizing *when you're fighting the platform* (replaced element, closed shadow tree) versus when you're just missing a class is a skill that saves hours of guessing.

02 Record<K,V> as an exhaustive lookup table

● PATTERN

KEYWORDS

Record<K,V>

exhaustiveness checking

union type

— WHAT

A TypeScript utility type describing an object required to have exactly one value for *every* key in a given type — used in `Badge` to map each difficulty level to its color.

— HOW

```
const difficultyColor: Record<Difficulty, ColorVariant> = {  
  beginner: "green",  
  intermediate: "gold",  
  advanced: "orange",  
  pro: "red",  
};  
  
const color = difficultyColor[level]; // bracket-notation lookup
```

— WHY

Type safety on **both** sides at once. TypeScript forces every key of `Difficulty` to have an entry — miss one and it's a compile error, not a silently-missing color at runtime. And `difficultyColor[level]` is guaranteed to return a valid `ColorVariant`, never `undefined`. A plain object, or a `switch` with no `default`, gives you neither guarantee.

◆ IDEA TO CONSIDER

Exhaustiveness checking is bigger than `Record` — TypeScript can enforce "every case handled" on `switch` statements over union types too (pair it with a `never`-typed default branch). Anywhere you're mapping a closed set of values to something else, ask whether the compiler is actually watching for a missed case, or just hoping you remembered.

03 Optional props for "one component, multiple roles"

● PATTERN

KEYWORDS

optional prop

presence as a signal

single responsibility

— WHAT

Making a prop optional so one component can serve two jobs, distinguished by whether that prop was passed.

— HOW

```
interface BadgeProps { label: string; level?: Difficulty; }

<Badge label="PARK" />                                {/* plain type badge */}
<Badge label="BEGINNER" level="beginner" />           {/* color-coded badge */}
```

— WHY

The two badges are identical in every way except one small, sometimes-absent piece of data. Splitting into two components would duplicate the box styling for no reason; a pair of required booleans would be clunkier than one prop whose presence already carries the meaning.

◆ IDEA TO CONSIDER

Watch for components that accumulate four or five optional props with unclear relationships to each other — that's often a sign the component is quietly doing *two unrelated jobs*, not one job with options. The tell here was narrow: exactly one prop, and its presence/absence maps to exactly one real distinction. Beyond that, it's usually cleaner to split.

04 Derive a value — don't accept it as a second prop

● PATTERN

● BUG

KEYWORDS

single source of truth

derived state

— WHAT

Computing a value *inside* a component from another prop, instead of accepting both as independent props the caller has to keep in sync.

— HOW

`Badge` went through a raw `color: string` prop, then a typed `color: ColorVariant` prop passed *alongside* `level`, before landing on deriving it: `const color = difficultyColor[level];` — computed internally, never accepted from outside.

— WHY

Two props that always have to agree are two sources of truth. TypeScript checks each prop's *type* independently, not whether they're *consistent* with each other — so `level="beginner"` paired with `color="red"` would type-check fine and just be wrong. Deriving one from the other removes the possibility entirely.

◆ IDEA TO CONSIDER

This is the same failure mode as an un-normalized database table (a customer's name stored in two places that can drift apart), or a Redux store that keeps a derived total in state instead of computing it from the line items. **Single source of truth** isn't a React idea or a CSS idea — it's one of the most load-bearing ideas in software architecture generally, and it shows up at every scale.

05 Computed CSS Modules class lookup

● CONCEPT

● GOTCHA

KEYWORDS

bracket notation

CSS Modules

runtime string matching

— WHAT

Picking a CSS Modules class at runtime using a variable, instead of a hardcoded `styles.someName`.

— HOW

```
styles.color          // hardcoded – always the same key
styles[color]         // computed – "color" is a runtime string like "green"
```

The computed form only works if the runtime string's possible values exactly match real class names in the module (`.green` , `.gold` , etc.).

— WHY

This is what makes `Record` (Ch. 02) actually reach the screen — the dictionary picks the *name*, the computed lookup turns the name into a real class. Break the naming match (classes named by level, while the variable holds a color) and the lookup silently returns `undefined` — this was the exact bug hit partway through building `Badge`.

◆ IDEA TO CONSIDER

Whenever a lookup crosses a language/file boundary — a JS string reaching into a CSS Modules object, a route param matching a file name, an env var name matching a config schema — nothing forces the two sides to stay in sync automatically. TypeScript guaranteed the `Record` was complete; it had *no idea* whether the CSS file actually defined matching classes. Naming mismatches across that kind of seam are a very common bug source, precisely because each side looks correct in isolation.

06 Descendant selectors for state-driven styling

● PATTERN

KEYWORDS `descendant selector` `state ownership` `cascade`

— WHAT

Putting a state class (`.confirmed` / `.unconfirmed`) on a *parent* element, and styling its children through descendant selectors, rather than repeating the state class on each child.

— HOW

```
<div className={` ${styles.root} ${indicator.className}`}>
  <div className={styles.circle}>...</div>
  <span className={styles.label}>...</span>
</div>
```

```
.confirmed .circle { background: oklch(0.72 0.16 150); }
.confirmed .label { color: oklch(0.72 0.16 150); }
```

— WHY

The circle and the label need *different* CSS properties for the same state (background/border vs. text color). Applying one raw class directly to both would leak properties across — a `background` meant for the circle would paint an unwanted box behind the label's text. Attaching state to the shared parent keeps each state defined once while still styling multiple children correctly.

◆ IDEA TO CONSIDER

This is really the same question as "where should this state live?" in React (the classic *lift state up* advice) — should the state live on the leaf that changes, or on the ancestor that all the changed leaves share? CSS and component state disagree on mechanics, but they're answering the identical architectural question: find the smallest common ancestor of everything that needs to react together.

07 Identity props vs. layout props

● PATTERN

KEYWORDS

component API design

inline style

design tokens

— WHAT

Two categories of prop that look similar but do different jobs: props describing what a component *is* (identity), and props describing how it behaves *in its current parent*

(layout).

— HOW

```
<Button label="Save Spot" variant="primary" grow={1.3} />
<Button label="Cancel"      variant="outline" grow={1}   />
```

`variant` selects a CSS Modules class — identity. `grow` is applied as an inline `style={{ flex: grow }}` — layout, because flex-grow only means something relative to sibling elements in a specific row.

— WHY

The test: *"if I moved this component to a different parent, would this prop still make sense?"* Identity survives the move — a primary button is still primary anywhere. Layout doesn't — flex-grow is meaningless outside a flex container. Identity belongs in the component's own CSS; layout belongs to whoever is placing it.

◆ IDEA TO CONSIDER

Every mature component library (Radix, MUI, shadcn/ui) draws this exact line, usually described as "the component owns its own look; the consumer owns its own position."

`variant` / `size` / `color` are identity props almost everywhere;

`margin` / `flex` / `width` are almost always left to the consumer, or handled through a separate layout primitive (a `<Stack>`, a `<Box>`) rather than baked into the button itself.

08 Generic components

● CONCEPT

KEYWORDS `generics` `type parameter` `type inference`

— WHAT

A component whose prop types aren't fixed — they're parameterized, so the same component works with different specific types while TypeScript still checks each usage correctly.

— HOW

```
function ChipGroup<T extends string>({ name, options }: ChipGroupProps<T>) { ... }  
  
<ChipGroup options={spotTypes} />  {/* T inferred as SpotType */}  
<ChipGroup options={features} />    {/* T inferred as Feature */}
```

`T` is a placeholder type, constrained to "must be some kind of string." TypeScript infers what `T` actually is at each call site.

— WHY

Without generics, `ChipGroup` would need `options: string[]` (loses domain typing entirely — no autocomplete, typos allowed) or two near-identical components, one typed for `SpotType[]` and one for `Feature[]` (duplication again). Generics let one component stay strictly typed against *whatever* specific union it's handed.

◆ IDEA TO CONSIDER

This is the same "too loose vs. too duplicated" trade-off you'll meet in typed API clients (`fetch<ResponseType>()`), typed form libraries, and any container/wrapper component (a `List<T>`, a `Select<T>`). The shape of the container repeats; only the payload type changes. Recognizing that shape is what tells you "this wants a type parameter" instead of "this wants a rewrite."

09 Native checkbox + :has() for zero-JS interactivity

● PATTERN

KEYWORDS

`:has()`

progressive enhancement

native form state

— WHAT

Getting a visually-toggleable UI element (a filter chip that turns gold when clicked) with no `useState`, no `onClick`, no event handler at all.

— HOW

```
<label className={styles.pill}>
  <input type="checkbox" defaultChecked />  {/* visually hidden */}
  PARK
</label>
```

```
.pill:has(input:checked) {
  background: var(--color-accent);
  color: var(--color-bg-nav);
}
```

— WHY

The browser handles checked/unchecked natively on click — no JS involved. `:has()` lets CSS read that native state directly, so the chip visually responds to clicks while the component still carries zero React state — consistent with this project's "static markup first, real interactivity later" build order, without looking broken in the meantime.

◆ IDEA TO CONSIDER

The general principle — prefer the platform's built-in state before reaching for framework state — is called **progressive enhancement**, and it's worth actively looking for. But know the real limit here: `:has()` only produces a *visual* side-effect. The moment anything else in the app needs to *read* "which filters are active" (to actually filter the map's spots), that state has to move into something JS can access — this trick buys time, it doesn't replace real state forever.

10 Choosing the right HTML element for plain text

● CONCEPT

KEYWORDS

semantic HTML

<label> vs

<data>

— WHAT

Deciding what tag to wrap non-interactive display text in — a spot's name, a distance readout, a badge's label.

— HOW

CONTENT	RIGHT ELEMENT	WHY NOT THE ALTERNATIVE
Short inline text, no role	<code></code>	carries zero implied behavior — exactly right for "this is just text"
A card's heading text	<code><h3></code>	not <code><label></code> — nothing here is a form control
A value something will read programmatically	<code><data value="0.4">0.4</data></code>	usually overkill unless the value is actually consumed elsewhere

— WHY

`<label>` carries real behavior — clicking it focuses or activates an *associated form control*. Using it on `SpotDetailCard`'s name field (a lingering leftover in the codebase) implies a relationship to a form input that doesn't exist. `<span>` implies nothing, which is exactly correct for plain display text.

◆ IDEA TO CONSIDER

Semantic HTML isn't decoration — screen readers, browser autofill, and search engines all read the *tag*, not the CSS sitting on top of it. A useful habit before reaching for a generic `<div>` / `<span>`: ask "does an element with the right native meaning already exist?" Nine times out of ten in a typical UI, one does.

A Glossary

Every keyword from the chapters above, in one place.

Replaced element

An HTML element (like `<select>`, `<video>`, `<input>`) whose rendering the browser controls internally — you can't insert child elements to alter its rendered content the way you can with a normal element.

Record<K, V>

A TypeScript utility type for an object that must have exactly one `V` for every value of the key type `K`. Gives compile-time exhaustiveness.

Exhaustiveness checking

The compiler verifying every possible case of a union/key type is handled somewhere, so an unhandled case is a build error, not a runtime surprise.

Optional prop

A component prop marked with `?` — its *presence or absence* can itself carry meaning, not just its value.

Single source of truth

The principle that any given fact should be stored/computed in exactly one place; everything else derives from it, so it can't drift out of sync.

Bracket notation (computed lookup)

`obj[variable]` — accessing an object property using a runtime value instead of a hardcoded name. Powerful, but only as safe as the guarantee that the key actually exists.

Descendant selector

A CSS selector like `.confirmed.circle` that matches an element only when it's nested inside another matching element — lets one class on a parent drive styling on several different children.

Identity vs. layout props

A component-API distinction: props that describe what a component *is* (belongs in its own CSS) vs. props that describe its relationship to its current parent (usually applied inline or by the consumer).

Generics (type parameter)

A placeholder type (like `T`) on a function or component, letting it stay strictly typed against whatever specific type it's given at each call site.

:has()

A CSS pseudo-class that matches an element if it *contains* something matching the given selector — e.g.

`.pill:has(input:checked)` styles the label based on its child checkbox's state.

Progressive enhancement

Building on top of what the platform (HTML/CSS/the browser) already does natively, before reaching for JavaScript to reproduce the same behavior.

Semantic HTML

Choosing elements for what they natively *mean* (a heading, a label, a button) rather than only for how they can be styled — affects accessibility, autofill, and SEO, not just appearance.

B Self-test

Answer aloud *before* opening each — explaining the "why" is what makes it stick.

Q1 Why can't you put a `<span>` arrow inside a `<select>` the way you would `<div>` ? ▶

Q2 Why does `Record<Difficulty, ColorVariant>` catch a missing-case bug at compile time that a `switch` with `default` wouldn't? ▶

Q3 What made `level` a good candidate for an optional prop, `Badge` into two components? ▶

Q4 What's wrong with a `level` and `color` as separate, individually well-typed, required props? ▶

Q5 Why did `styles[color]` silently return `undefined` partway through building `Badge` ? ▶

Q6 Why did `Indicator` put its state class on the root `<div>` instead of on the circle and the label separately? ▶

Q7 Why does `Button` 's `grow` prop live as an inline style instead of becoming another `variant` ? ▶

Q8 Why does `ChipGroup<T extends String>` avoid duplicating `SpotType[]` and `Feature[]` ? ▶

Q9 Why can't the filter `:has()` -driven toggle actually filter the map's spots by chips' itself? ▶

Q10 Why is `<label>` the wrong element for a card's heading text? ▶

C Study links — go deeper

- ↗ [TypeScript — Record<K,V> utility type](#)
- ↗ [TypeScript — Generics handbook](#)
- ↗ [TypeScript — narrowing & exhaustiveness patterns](#)
- ↗ [MDN — :has\(\) pseudo-class](#)
- ↗ [MDN — Replaced elements](#)
- ↗ [MDN — the <data> element](#)
- ↗ [React — Lifting state up](#)
- ↗ [React — Passing props to a component](#)

